SIGCB-QR

Sistema de Gestion de Biblioteca

CODIGO FUENTE

Entregable Tercera Entrega - v0.9.0-rc

Julio 2026

Archivos incluidos

1. sigcb-qr-api\src\main\java\com\sigcbqr\security\JwtTokenProvider.java
2. sigcb-qr-api\src\main\java\com\sigcbqr\security\JwtAuthenticationFilter.java
3. sigcb-qr-api\src\main\java\com\sigcbqr\config\SecurityConfig.java
4. sigcb-qr-api\src\main\java\com\sigcbqr\security\JwtBlacklistService.java
5. sigcb-qr-api\src\main\java\com\sigcbqr\controller\AuthController.java
6. sigcb-qr-api\src\main\java\com\sigcbqr\service\AuthService.java
7. sigcb-qr-api\src\main\java\com\sigcbqr\controller\PrestamoController.java
8. sigcb-qr-api\src\main\java\com\sigcbqr\service\PrestamoService.java
9. sigcb-qr-api\src\main\java\com\sigcbqr\config\CacheConfig.java
10. sigcb-qr-api\src\main\java\com\sigcbqr\exception\GlobalExceptionHandler.java
11. sigcb-qr-api\src\main\java\com\sigcbqr\model\entity\Prestamo.java
12. sigcb-qr-api\src\main\java\com\sigcbqr\model\entity\Libro.java
13. sigcb-qr-api\src\main\java\com\sigcbqr\model\entity\Usuario.java
14. db\procs\sp_crear_prestamo.sql
15. db\procs\sp_devolver_prestamo.sql
16. db\procs\sp_reporte_prestamos_diarios.sql

Total: 16 archivos

sigcb-qr-api\src\main\java\com\sigcbqr\security\JwtTokenProvider.java

```java
package com.sigcbqr.security;

import io.jsonwebtoken.*;
import io.jsonwebtoken.io.Decoders;
import io.jsonwebtoken.security.Keys;
import jakarta.servlet.http.Cookie;
import jakarta.servlet.http.HttpServletRequest;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;

import javax.crypto.SecretKey;
import java.util.Date;
import java.util.UUID;

@Component
public class JwtTokenProvider {

    private final SecretKey secretKey;
    private final long expirationMs;
    private final String issuer;
    private final String audience;
    private final boolean secureCookie;
    private final JwtBlacklistService blacklistService;

    public JwtTokenProvider(
            @Value("${app.jwt.secret}") String secret,
            @Value("${app.jwt.expiration-ms}") long expirationMs,
            @Value("${app.jwt.issuer}") String issuer,
            @Value("${app.jwt.audience}") String audience,
            @Value("${app.jwt.secure-cookie}") boolean secureCookie,
            JwtBlacklistService blacklistService) {
        this.secretKey = Keys.hmacShaKeyFor(Decoders.BASE64.decode(secret));
        this.expirationMs = expirationMs;
        this.issuer = issuer;
        this.audience = audience;
        this.secureCookie = secureCookie;
        this.blacklistService = blacklistService;
    }

    public String generateToken(Long userId, String email, String rol) {
        Date now = new Date();
        Date expiryDate = new Date(now.getTime() + expirationMs);

        return Jwts.builder()
                .id(UUID.randomUUID().toString())
                .issuer(issuer)
                .subject(userId.toString())
                .audience().add(audience).and()
                .claim("email", email)
                .claim("rol", rol)
                .issuedAt(now)
                .notBefore(now)
                .expiration(expiryDate)
                .signWith(secretKey)
                .compact();
    }

    public Long getUserIdFromToken(String token) {
        Claims claims = parseToken(token);
        return Long.parseLong(claims.getSubject());
    }

    public String getEmailFromToken(String token) {
        Claims claims = parseToken(token);
        return claims.get("email", String.class);
    }

    public String getRolFromToken(String token) {
        Claims claims = parseToken(token);
        return claims.get("rol", String.class);
    }

```

sigcb-qr-api\src\main\java\com\sigcbqr\security\JwtTokenProvider.java (cont.)

```
    public String getJtiFromToken(String token) {
        Claims claims = parseToken(token);
        return claims.getId();
    }

    public boolean validateToken(String token) {
        try {
            Claims claims = parseToken(token);
            if (blacklistService.isBlacklisted(claims.getId())) {
                return false;
            }
            return true;
        } catch (JwtException | IllegalArgumentException e) {
            return false;
        }
    }

    private Claims parseToken(String token) {
        return Jwts.parser()
                .verifyWith(secretKey)
                .build()
                .parseSignedClaims(token)
                .getPayload();
    }

    public String extractTokenFromCookie(HttpServletRequest request) {
        Cookie[] cookies = request.getCookies();
        if (cookies != null) {
            for (Cookie cookie : cookies) {
                if ("access_token".equals(cookie.getName())) {
                    return cookie.getValue();
                }
            }
        }
        return null;
    }

    public Cookie createAccessTokenCookie(String token) {
        Cookie cookie = new Cookie("access_token", token);
        cookie.setHttpOnly(true);
        cookie.setSecure(secureCookie);
        cookie.setPath("/");
        cookie.setMaxAge((int) (expirationMs / 1000));
        cookie.setAttribute("SameSite", "Strict");
        return cookie;
    }

    public Date getExpirationFromToken(String token) {
        return parseToken(token).getExpiration();
    }

    public SecretKey getSecretKey() {
        return secretKey;
    }

    public JwtBlacklistService getJwtBlacklistService() {
        return blacklistService;
    }

    public Cookie createLogoutCookie() {
        Cookie cookie = new Cookie("access_token", null);
        cookie.setHttpOnly(true);
        cookie.setSecure(secureCookie);
        cookie.setPath("/");
        cookie.setMaxAge(0);
        cookie.setAttribute("SameSite", "Strict");
        return cookie;
    }
}
```

sigcb-qr-api\src\main\java\com\sigcbqr\security\JwtAuthenticationFilter.java

```java
package com.sigcbqr.security;

import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.web.authentication.WebAuthenticationDetailsSource;
import org.springframework.stereotype.Component;
import org.springframework.util.StringUtils;
import org.springframework.web.filter.OncePerRequestFilter;

import java.io.IOException;
import java.util.List;

@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private final JwtTokenProvider tokenProvider;

    public JwtAuthenticationFilter(JwtTokenProvider tokenProvider) {
        this.tokenProvider = tokenProvider;
    }

    @Override
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain) throws ServletException, IOException {

        String token = tokenProvider.extractTokenFromCookie(request);

        if (StringUtils.hasText(token) && tokenProvider.validateToken(token)) {
            Long userId = tokenProvider.getUserIdFromToken(token);
            String email = tokenProvider.getEmailFromToken(token);
            String rol = tokenProvider.getRolFromToken(token);

            List<SimpleGrantedAuthority> authorities = List.of(
                    new SimpleGrantedAuthority("ROLE_" + rol)
            );

            UserPrincipal userPrincipal = new UserPrincipal(userId, email, null, rol, authorities);

            UsernamePasswordAuthenticationToken authentication =
                    new UsernamePasswordAuthenticationToken(userPrincipal, null, authorities);
            authentication.setDetails(new WebAuthenticationDetailsSource().buildDetails(request));

            SecurityContextHolder.getContext().setAuthentication(authentication);
        }

        filterChain.doFilter(request, response);
    }
}
```

sigcb-qr-api\src\main\java\com\sigcbqr\config\SecurityConfig.java

```java
package com.sigcbqr.config;

import com.sigcbqr.security.JwtAuthenticationEntryPoint;
import com.sigcbqr.security.JwtAuthenticationFilter;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.config.annotation.authentication.configuration.AuthenticationConfiguration;
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;

@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig {

    private final JwtAuthenticationFilter jwtAuthenticationFilter;
    private final JwtAuthenticationEntryPoint jwtAuthenticationEntryPoint;

    public SecurityConfig(JwtAuthenticationFilter jwtAuthenticationFilter,
                          JwtAuthenticationEntryPoint jwtAuthenticationEntryPoint) {
        this.jwtAuthenticationFilter = jwtAuthenticationFilter;
        this.jwtAuthenticationEntryPoint = jwtAuthenticationEntryPoint;
    }

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .csrf(csrf -> csrf.disable())
            .cors(cors -> {})
            .exceptionHandling(ex -> ex.authenticationEntryPoint(jwtAuthenticationEntryPoint))
            .sessionManagement(session -> session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/auth/**").permitAll()
                .requestMatchers("/swagger-ui/**", "/api-docs/**", "/v3/api-docs/**").permitAll()
                .requestMatchers("/api/dashboard/**").authenticated()
                .requestMatchers("/api/**").authenticated()
                .anyRequest().permitAll()
            )
            .addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public AuthenticationManager authenticationManager(AuthenticationConfiguration config) throws Exception {
        return config.getAuthenticationManager();
    }
}
```

sigcb-qr-api\src\main\java\com\sigcbqr\security\JwtBlacklistService.java

```java
package com.sigcbqr.security;

import com.sigcbqr.model.entity.JwtBlacklist;
import com.sigcbqr.repository.JwtBlacklistRepository;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.time.LocalDateTime;

@Service
public class JwtBlacklistService {

    private final JwtBlacklistRepository repository;

    public JwtBlacklistService(JwtBlacklistRepository repository) {
        this.repository = repository;
    }

    @Transactional
    public void blacklist(String jti, LocalDateTime expiration) {
        if (!repository.existsByJti(jti)) {
            JwtBlacklist entry = JwtBlacklist.builder()
                    .jti(jti)
                    .fechaExpiracion(expiration)
                    .build();
            repository.save(entry);
        }
    }

    public boolean isBlacklisted(String jti) {
        return repository.existsByJti(jti);
    }

    @Scheduled(cron = "0 0 */6 * * *")
    @Transactional
    public void cleanExpiredEntries() {
        repository.deleteByFechaExpiracionBefore(LocalDateTime.now());
    }
}
```

sigcb-qr-api\src\main\java\com\sigcbqr\controller\AuthController.java

```java
package com.sigcbqr.controller;

import com.sigcbqr.model.dto.request.LoginRequest;
import com.sigcbqr.model.dto.request.RegisterRequest;
import com.sigcbqr.model.dto.response.ApiResponse;
import com.sigcbqr.model.dto.response.LoginResponse;
import com.sigcbqr.model.dto.response.UsuarioResponse;
import com.sigcbqr.security.JwtTokenProvider;
import com.sigcbqr.security.UserPrincipal;
import com.sigcbqr.service.AuthService;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.validation.Valid;
import org.springframework.http.ResponseEntity;
import org.springframework.security.core.annotation.AuthenticationPrincipal;
import org.springframework.web.bind.annotation.*;

import java.time.LocalDateTime;
import java.util.Date;

@RestController
@RequestMapping("/api/auth")
@Tag(name = "Autenticación", description = "Endpoints de autenticación con JWT en cookie HttpOnly")
public class AuthController {

    private final AuthService authService;
    private final JwtTokenProvider tokenProvider;

    public AuthController(AuthService authService, JwtTokenProvider tokenProvider) {
        this.authService = authService;
        this.tokenProvider = tokenProvider;
    }

    @PostMapping("/login")
    @Operation(summary = "Iniciar sesión", description = "Autentica al usuario y devuelve un JWT en cookie HttpOnly")
    public ResponseEntity<ApiResponse> login(@Valid @RequestBody LoginRequest request,
                                             HttpServletResponse response) {
        LoginResponse loginResponse = authService.login(request);

        String token = tokenProvider.generateToken(
                loginResponse.getId(),
                loginResponse.getEmail(),
                loginResponse.getRol()
        );

        var cookie = tokenProvider.createAccessTokenCookie(token);
        response.addCookie(cookie);

        return ResponseEntity.ok(ApiResponse.success("Inicio de sesión exitoso", loginResponse));
    }

    @PostMapping("/register")
    @Operation(summary = "Registrar usuario", description = "Registra un nuevo usuario y devuelve JWT en cookie")
    public ResponseEntity<ApiResponse> register(@Valid @RequestBody RegisterRequest request,
                                                HttpServletResponse response) {
        LoginResponse loginResponse = authService.register(request);

        String token = tokenProvider.generateToken(
                loginResponse.getId(),
                loginResponse.getEmail(),
                loginResponse.getRol()
        );

        var cookie = tokenProvider.createAccessTokenCookie(token);
        response.addCookie(cookie);

        return ResponseEntity.ok(ApiResponse.success("Registro exitoso", loginResponse));
    }

    @PostMapping("/logout")
```

sigcb-qr-api\src\main\java\com\sigcbqr\controller\AuthController.java (cont.)

```
73      @Operation(summary = "Cerrar sesión", description = "Invalida el JWT actual agregando su JTI a la blacklist")
74      public ResponseEntity<ApiResponse> logout(HttpServletRequest request,
75                                                HttpServletResponse response) {
76          String token = tokenProvider.extractTokenFromCookie(request);
77          if (token != null && tokenProvider.validateToken(token)) {
78              Date expiration = tokenProvider.getExpirationFromToken(token);
79              authService.logout(token, new java.sql.Timestamp(expiration.getTime()).toLocalDateTime());
80          }
81
82          var cookie = tokenProvider.createLogoutCookie();
83          response.addCookie(cookie);
84
85          return ResponseEntity.ok(ApiResponse.success("Sesión cerrada exitosamente", null));
86      }
87
88      @GetMapping("/me")
89      @Operation(summary = "Usuario actual", description = "Obtiene los datos del usuario autenticado")
90      public ResponseEntity<ApiResponse> me(@AuthenticationPrincipal UserPrincipal userPrincipal) {
91          var usuario = authService.getCurrentUser(userPrincipal.id());
92          var response = UsuarioResponse.builder()
93                  .id(usuario.getId())
94                  .nombre(usuario.getNombre())
95                  .email(usuario.getEmail())
96                  .telefono(usuario.getTelefono())
97                  .rol(usuario.getRol().getNombre())
98                  .activo(usuario.getActivo())
99                  .createdAt(usuario.getCreatedAt())
100                 .build();
101         return ResponseEntity.ok(ApiResponse.success("Usuario actual", response));
102     }
103 }
```

sigcb-qr-api\src\main\java\com\sigcbqr\service\AuthService.java

```java
package com.sigcbqr.service;

import com.sigcbqr.exception.BadRequestException;
import com.sigcbqr.exception.ResourceNotFoundException;
import com.sigcbqr.model.dto.request.LoginRequest;
import com.sigcbqr.model.dto.request.RegisterRequest;
import com.sigcbqr.model.dto.response.LoginResponse;
import com.sigcbqr.model.entity.Rol;
import com.sigcbqr.model.entity.Usuario;
import com.sigcbqr.repository.RolRepository;
import com.sigcbqr.repository.UsuarioRepository;
import com.sigcbqr.security.JwtTokenProvider;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.time.LocalDateTime;

@Service
public class AuthService {

    private final AuthenticationManager authenticationManager;
    private final JwtTokenProvider tokenProvider;
    private final UsuarioRepository usuarioRepository;
    private final RolRepository rolRepository;
    private final PasswordEncoder passwordEncoder;

    public AuthService(AuthenticationManager authenticationManager,
                       JwtTokenProvider tokenProvider,
                       UsuarioRepository usuarioRepository,
                       RolRepository rolRepository,
                       PasswordEncoder passwordEncoder) {
        this.authenticationManager = authenticationManager;
        this.tokenProvider = tokenProvider;
        this.usuarioRepository = usuarioRepository;
        this.rolRepository = rolRepository;
        this.passwordEncoder = passwordEncoder;
    }

    public LoginResponse login(LoginRequest request) {
        authenticationManager.authenticate(
                new UsernamePasswordAuthenticationToken(request.getEmail(), request.getPassword())
        );

        Usuario usuario = usuarioRepository.findByEmail(request.getEmail())
                .orElseThrow(() -> new ResourceNotFoundException("Usuario no encontrado"));

        String token = tokenProvider.generateToken(usuario.getId(), usuario.getEmail(),
                usuario.getRol().getNombre());

        return LoginResponse.builder()
                .id(usuario.getId())
                .nombre(usuario.getNombre())
                .email(usuario.getEmail())
                .rol(usuario.getRol().getNombre())
                .mensaje("Inicio de sesión exitoso")
                .build();
    }

    @Transactional
    public LoginResponse register(RegisterRequest request) {
        if (usuarioRepository.existsByEmail(request.getEmail())) {
            throw new BadRequestException("El correo ya está registrado");
        }

        Rol rol;
        if (request.getRolId() != null) {
            rol = rolRepository.findById(request.getRolId())
                    .orElseThrow(() -> new ResourceNotFoundException("Rol no encontrado"));
        } else {
```

sigcb-qr-api\src\main\java\com\sigcbqr\service\AuthService.java (cont.)

```
            rol = rolRepository.findByNombre("ESTUDIANTE")
                    .orElseThrow(() -> new ResourceNotFoundException("Rol ESTUDIANTE no encontrado"));
        }

        Usuario usuario = Usuario.builder()
                .nombre(request.getNombre())
                .email(request.getEmail())
                .password(passwordEncoder.encode(request.getPassword()))
                .telefono(request.getTelefono())
                .activo(true)
                .rol(rol)
                .build();

        usuario = usuarioRepository.save(usuario);

        String token = tokenProvider.generateToken(usuario.getId(), usuario.getEmail(),
                usuario.getRol().getNombre());

        return LoginResponse.builder()
                .id(usuario.getId())
                .nombre(usuario.getNombre())
                .email(usuario.getEmail())
                .rol(usuario.getRol().getNombre())
                .mensaje("Registro exitoso")
                .build();
    }

    public void logout(String token, LocalDateTime expiration) {
        String jti = tokenProvider.getJtiFromToken(token);
        tokenProvider.getJwtBlacklistService().blacklist(jti, expiration);
    }

    public Usuario getCurrentUser(Long userId) {
        return usuarioRepository.findById(userId)
                .orElseThrow(() -> new ResourceNotFoundException("Usuario", userId));
    }
}
```

sigcb-qr-api\src\main\java\com\sigcbqr\controller\PrestamoController.java

```java
package com.sigcbqr.controller;

import com.sigcbqr.model.dto.request.PrestamoRequest;
import com.sigcbqr.model.dto.response.ApiResponse;
import com.sigcbqr.model.dto.response.PageResponse;
import com.sigcbqr.model.dto.response.PrestamoResponse;
import com.sigcbqr.service.PrestamoService;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.validation.Valid;
import org.springframework.data.domain.Pageable;
import org.springframework.data.web.PageableDefault;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/prestamos")
@Tag(name = "Préstamos", description = "Gestión de préstamos, devoluciones y renovaciones")
public class PrestamoController {

    private final PrestamoService prestamoService;

    public PrestamoController(PrestamoService prestamoService) {
        this.prestamoService = prestamoService;
    }

    @GetMapping
    @Operation(summary = "Listar préstamos", description = "Obtiene todos los préstamos con paginación")
    @PreAuthorize("hasAnyRole('ADMIN', 'BIBLIOTECARIO')")
    public ResponseEntity<PageResponse<PrestamoResponse>> listar(
            @PageableDefault(size = 10, sort = "fechaPrestamo") Pageable pageable) {
        var page = prestamoService.listar(pageable);
        return ResponseEntity.ok(PageResponse.from(page));
    }

    @GetMapping("/usuario/{usuarioId}")
    @Operation(summary = "Préstamos por usuario", description = "Obtiene los préstamos de un usuario específico")
    public ResponseEntity<PageResponse<PrestamoResponse>> listarPorUsuario(
            @PathVariable Long usuarioId,
            @PageableDefault(size = 10) Pageable pageable) {
        var page = prestamoService.listarPorUsuario(usuarioId, pageable);
        return ResponseEntity.ok(PageResponse.from(page));
    }

    @GetMapping("/{id}")
    @Operation(summary = "Obtener préstamo", description = "Obtiene un préstamo por su ID")
    public ResponseEntity<ApiResponse> obtener(@PathVariable Long id) {
        var prestamo = prestamoService.obtener(id);
        return ResponseEntity.ok(ApiResponse.success("Préstamo encontrado", prestamo));
    }

    @PostMapping
    @PreAuthorize("hasAnyRole('ADMIN', 'BIBLIOTECARIO')")
    @Operation(summary = "Crear préstamo", description = "Registra un nuevo préstamo")
    public ResponseEntity<ApiResponse> crear(@Valid @RequestBody PrestamoRequest request) {
        var prestamo = prestamoService.crear(request);
        return ResponseEntity.ok(ApiResponse.created("Préstamo registrado", prestamo));
    }

    @PutMapping("/{id}/devolver")
    @PreAuthorize("hasAnyRole('ADMIN', 'BIBLIOTECARIO')")
    @Operation(summary = "Devolver libro", description = "Registra la devolución de un préstamo")
    public ResponseEntity<ApiResponse> devolver(@PathVariable Long id) {
        var prestamo = prestamoService.devolver(id);
        return ResponseEntity.ok(ApiResponse.success("Devolución registrada", prestamo));
    }

    @PutMapping("/{id}/renovar")
    @PreAuthorize("hasAnyRole('ADMIN', 'BIBLIOTECARIO')")
    @Operation(summary = "Renovar préstamo", description = "Renueva un préstamo activo")
    public ResponseEntity<ApiResponse> renovar(@PathVariable Long id) {
```

sigcb-qr-api\src\main\java\com\sigcbqr\controller\PrestamoController.java (cont.)

```
        var prestamo = prestamoService.renovar(id);
        return ResponseEntity.ok(ApiResponse.success("Préstamo renovado", prestamo));
    }
}
```

sigcb-qr-api\src\main\java\com\sigcbqr\service\PrestamoService.java

```java
package com.sigcbqr.service;

import com.sigcbqr.exception.BadRequestException;
import com.sigcbqr.exception.ResourceNotFoundException;
import com.sigcbqr.model.dto.request.PrestamoRequest;
import com.sigcbqr.model.dto.response.PrestamoResponse;
import com.sigcbqr.model.entity.*;
import com.sigcbqr.repository.*;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.time.LocalDateTime;

@Service
public class PrestamoService {

    private final PrestamoRepository prestamoRepository;
    private final UsuarioRepository usuarioRepository;
    private final InventarioRepository inventarioRepository;
    private final LibroRepository libroRepository;

    public PrestamoService(PrestamoRepository prestamoRepository,
                           UsuarioRepository usuarioRepository,
                           InventarioRepository inventarioRepository,
                           LibroRepository libroRepository) {
        this.prestamoRepository = prestamoRepository;
        this.usuarioRepository = usuarioRepository;
        this.inventarioRepository = inventarioRepository;
        this.libroRepository = libroRepository;
    }

    public Page<PrestamoResponse> listar(Pageable pageable) {
        return prestamoRepository.findAll(pageable).map(this::toResponse);
    }

    public Page<PrestamoResponse> listarPorUsuario(Long usuarioId, Pageable pageable) {
        return prestamoRepository.findByUsuarioIdOrderByFechaPrestamoDesc(usuarioId, pageable)
                .map(this::toResponse);
    }

    public PrestamoResponse obtener(Long id) {
        Prestamo prestamo = prestamoRepository.findById(id)
                .orElseThrow(() -> new ResourceNotFoundException("Préstamo", id));
        return toResponse(prestamo);
    }

    @Transactional
    public PrestamoResponse crear(PrestamoRequest request) {
        Usuario usuario = usuarioRepository.findById(request.getUsuarioId())
                .orElseThrow(() -> new ResourceNotFoundException("Usuario", request.getUsuarioId()));

        Inventario inventario = inventarioRepository.findById(request.getInventarioId())
                .orElseThrow(() -> new ResourceNotFoundException("Ejemplar", request.getInventarioId()));

        if (!"DISPONIBLE".equals(inventario.getEstado())) {
            throw new BadRequestException("El ejemplar no está disponible");
        }

        long prestamosActivos = prestamoRepository.countByUsuarioIdAndEstado(request.getUsuarioId(), "ACTIVO");
        if (prestamosActivos >= 5) {
            throw new BadRequestException("El usuario tiene demasiados préstamos activos");
        }

        Prestamo prestamo = Prestamo.builder()
                .usuario(usuario)
                .inventario(inventario)
                .fechaPrestamo(LocalDateTime.now())
                .fechaVencimiento(LocalDateTime.now().plusDays(7))
                .estado("ACTIVO")
                .build();
```

sigcb-qr-api\src\main\java\com\sigcbqr\service\PrestamoService.java (cont.)

```
73
74          inventario.setEstado("PRESTADO");
75          inventarioRepository.save(inventario);
76
77          Libro libro = inventario.getLibro();
78          libro.setEjemplaresDisponibles(libro.getEjemplaresDisponibles() - 1);
79          libroRepository.save(libro);
80
81          prestamo = prestamoRepository.save(prestamo);
82          return toResponse(prestamo);
83      }
84
85      @Transactional
86      public PrestamoResponse devolver(Long id) {
87          Prestamo prestamo = prestamoRepository.findById(id)
88                  .orElseThrow(() -> new ResourceNotFoundException("Préstamo", id));
89
90          if ("DEVUELTO".equals(prestamo.getEstado())) {
91              throw new BadRequestException("El préstamo ya fue devuelto");
92          }
93
94          prestamo.setFechaDevolucion(LocalDateTime.now());
95          prestamo.setEstado("DEVUELTO");
96
97          Inventario inventario = prestamo.getInventario();
98          inventario.setEstado("DISPONIBLE");
99          inventarioRepository.save(inventario);
100
101         Libro libro = inventario.getLibro();
102         libro.setEjemplaresDisponibles(libro.getEjemplaresDisponibles() + 1);
103         libroRepository.save(libro);
104
105         // Generar multa si está vencido
106         if (prestamo.getFechaVencimiento().isBefore(LocalDateTime.now())) {
107             // Multa lógica here - handled by MultaService
108         }
109
110         prestamo = prestamoRepository.save(prestamo);
111         return toResponse(prestamo);
112     }
113
114     @Transactional
115     public PrestamoResponse renovar(Long id) {
116         Prestamo prestamo = prestamoRepository.findById(id)
117                 .orElseThrow(() -> new ResourceNotFoundException("Préstamo", id));
118
119         if (!"ACTIVO".equals(prestamo.getEstado())) {
120             throw new BadRequestException("Solo se pueden renovar préstamos activos");
121         }
122
123         prestamo.setEstado("RENOVADO");
124         prestamo.setObservaciones("Renovado - nueva fecha: " + LocalDateTime.now().plusDays(7));
125         prestamoRepository.save(prestamo);
126
127         Prestamo nuevoPrestamo = Prestamo.builder()
128                 .usuario(prestamo.getUsuario())
129                 .inventario(prestamo.getInventario())
130                 .fechaPrestamo(LocalDateTime.now())
131                 .fechaVencimiento(LocalDateTime.now().plusDays(7))
132                 .estado("ACTIVO")
133                 .observaciones("Renovación del préstamo #" + prestamo.getId())
134                 .build();
135
136         nuevoPrestamo = prestamoRepository.save(nuevoPrestamo);
137         return toResponse(nuevoPrestamo);
138     }
139
140     private PrestamoResponse toResponse(Prestamo prestamo) {
141         return PrestamoResponse.builder()
142                 .id(prestamo.getId())
143                 .usuarioNombre(prestamo.getUsuario().getNombre())
144                 .libroTitulo(prestamo.getInventario().getLibro().getTitulo())
145                 .codigoEjemplar(prestamo.getInventario().getCodigoEjemplar())
```

sigcb-qr-api\src\main\java\com\sigcbqr\service\PrestamoService.java (cont.)

```
146                 .fechaPrestamo(prestamo.getFechaPrestamo())
147                 .fechaVencimiento(prestamo.getFechaVencimiento())
148                 .fechaDevolucion(prestamo.getFechaDevolucion())
149                 .estado(prestamo.getEstado())
150                 .observaciones(prestamo.getObservaciones())
151                 .build();
152     }
153 }
```

sigcb-qr-api\src\main\java\com\sigcbqr\config\CacheConfig.java

```java
package com.sigcbqr.config;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.cache.CacheManager;
import org.springframework.cache.annotation.EnableCaching;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.data.redis.cache.RedisCacheConfiguration;
import org.springframework.data.redis.cache.RedisCacheManager;
import org.springframework.data.redis.connection.RedisConnectionFactory;
import org.springframework.data.redis.serializer.GenericJackson2JsonRedisSerializer;
import org.springframework.data.redis.serializer.RedisSerializationContext;

import java.time.Duration;

@Configuration
@EnableCaching
public class CacheConfig {

    @Value("${app.cache.default-ttl-seconds}")
    private int defaultTtlSeconds;

    @Bean
    public CacheManager cacheManager(RedisConnectionFactory connectionFactory) {
        RedisCacheConfiguration config = RedisCacheConfiguration.defaultCacheConfig()
                .entryTtl(Duration.ofSeconds(defaultTtlSeconds))
                .disableCachingNullValues()
                .serializeValuesWith(
                        RedisSerializationContext.SerializationPair.fromSerializer(
                                new GenericJackson2JsonRedisSerializer()));

        return RedisCacheManager.builder(connectionFactory)
                .cacheDefaults(config)
                .build();
    }
}
```

sigcb-qr-api\src\main\java\com\sigcbqr\exception\GlobalExceptionHandler.java

```java
package com.sigcbqr.exception;

import org.springframework.http.HttpStatus;
import org.springframework.http.ProblemDetail;
import org.springframework.security.authentication.BadCredentialsException;
import org.springframework.validation.FieldError;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.net.URI;
import java.util.HashMap;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {

    public static final String BASE_ERROR_URL = "https://api.sigcbqr.com/errors";

    @ExceptionHandler(ResourceNotFoundException.class)
    public ProblemDetail handleNotFound(ResourceNotFoundException ex) {
        ProblemDetail pd = ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND, ex.getMessage());
        pd.setType(URI.create(BASE_ERROR_URL + "/not-found"));
        pd.setTitle("Recurso no encontrado");
        pd.setProperty("timestamp", System.currentTimeMillis());
        return pd;
    }

    @ExceptionHandler(BadRequestException.class)
    public ProblemDetail handleBadRequest(BadRequestException ex) {
        ProblemDetail pd = ProblemDetail.forStatusAndDetail(HttpStatus.BAD_REQUEST, ex.getMessage());
        pd.setType(URI.create(BASE_ERROR_URL + "/bad-request"));
        pd.setTitle("Solicitud inválida");
        pd.setProperty("timestamp", System.currentTimeMillis());
        return pd;
    }

    @ExceptionHandler(BadCredentialsException.class)
    public ProblemDetail handleBadCredentials(BadCredentialsException ex) {
        ProblemDetail pd = ProblemDetail.forStatusAndDetail(HttpStatus.UNAUTHORIZED, "Credenciales inválidas");
        pd.setType(URI.create(BASE_ERROR_URL + "/unauthorized"));
        pd.setTitle("No autorizado");
        pd.setProperty("timestamp", System.currentTimeMillis());
        return pd;
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ProblemDetail handleValidation(MethodArgumentNotValidException ex) {
        Map<String, String> errors = new HashMap<>();
        for (FieldError error : ex.getBindingResult().getFieldErrors()) {
            errors.put(error.getField(), error.getDefaultMessage());
        }
        ProblemDetail pd = ProblemDetail.forStatusAndDetail(HttpStatus.BAD_REQUEST, "Error de validación");
        pd.setType(URI.create(BASE_ERROR_URL + "/validation"));
        pd.setTitle("Error de validación");
        pd.setProperty("errors", errors);
        pd.setProperty("timestamp", System.currentTimeMillis());
        return pd;
    }

    @ExceptionHandler(Exception.class)
    public ProblemDetail handleGeneral(Exception ex) {
        ProblemDetail pd = ProblemDetail.forStatusAndDetail(HttpStatus.INTERNAL_SERVER_ERROR,
                "Error interno del servidor: " + ex.getMessage());
        pd.setType(URI.create(BASE_ERROR_URL + "/internal-error"));
        pd.setTitle("Error interno");
        pd.setProperty("timestamp", System.currentTimeMillis());
        return pd;
    }
}
```

sigcb-qr-api\src\main\java\com\sigcbqr\model\entity\Prestamo.java

```java
package com.sigcbqr.model.entity;

import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
import jakarta.persistence.*;
import lombok.*;

import java.time.LocalDateTime;

@Entity
@Table(name = "prestamos")
@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
@JsonIgnoreProperties({"hibernateLazyInitializer", "handler"})
public class Prestamo {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "usuario_id", nullable = false)
    private Usuario usuario;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "inventario_id", nullable = false)
    private Inventario inventario;

    @Column(name = "fecha_prestamo")
    private LocalDateTime fechaPrestamo;

    @Column(name = "fecha_vencimiento", nullable = false)
    private LocalDateTime fechaVencimiento;

    @Column(name = "fecha_devolucion")
    private LocalDateTime fechaDevolucion;

    @Column(length = 30)
    private String estado;

    @Column(columnDefinition = "TEXT")
    private String observaciones;

    @Column(name = "created_at")
    private LocalDateTime createdAt;

    @Column(name = "updated_at")
    private LocalDateTime updatedAt;

    @PrePersist
    protected void onCreate() {
        createdAt = LocalDateTime.now();
        updatedAt = LocalDateTime.now();
        if (estado == null) estado = "ACTIVO";
        if (fechaPrestamo == null) fechaPrestamo = LocalDateTime.now();
    }

    @PreUpdate
    protected void onUpdate() {
        updatedAt = LocalDateTime.now();
    }
}
```

sigcb-qr-api\src\main\java\com\sigcbqr\model\entity\Libro.java

```java
package com.sigcbqr.model.entity;

import jakarta.persistence.*;
import lombok.*;

import java.time.LocalDateTime;
import java.util.HashSet;
import java.util.Set;

@Entity
@Table(name = "libros")
@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
public class Libro {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 300)
    private String titulo;

    @Column(unique = true, length = 20)
    private String isbn;

    @Column(name = "anio_publicacion")
    private Integer anioPublicacion;

    @Column(length = 50)
    private String edicion;

    @Column(name = "ejemplares_totales")
    private Integer ejemplaresTotales;

    @Column(name = "ejemplares_disponibles")
    private Integer ejemplaresDisponibles;

    @Column(length = 100)
    private String ubicacion;

    @Column(columnDefinition = "TEXT")
    private String descripcion;

    private String portada;

    private Boolean activo;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "categoria_id")
    private Categoria categoria;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "editorial_id")
    private Editorial editorial;

    @ManyToMany(fetch = FetchType.LAZY)
    @JoinTable(
        name = "libro_autores",
        joinColumns = @JoinColumn(name = "libro_id"),
        inverseJoinColumns = @JoinColumn(name = "autor_id")
    )
    @Builder.Default
    private Set<Autor> autores = new HashSet<>();

    @Column(name = "created_at")
    private LocalDateTime createdAt;

    @Column(name = "updated_at")
    private LocalDateTime updatedAt;

    @PrePersist
    protected void onCreate() {
        createdAt = LocalDateTime.now();
```

sigcb-qr-api\src\main\java\com\sigcbqr\model\entity\Libro.java (cont.)

```
73          updatedAt = LocalDateTime.now();
74          if (activo == null) activo = true;
75          if (ejemplaresTotales == null) ejemplaresTotales = 1;
76          if (ejemplaresDisponibles == null) ejemplaresDisponibles = ejemplaresTotales;
77      }
78
79      @PreUpdate
80      protected void onUpdate() {
81          updatedAt = LocalDateTime.now();
82      }
83  }
```

sigcb-qr-api\src\main\java\com\sigcbqr\model\entity\Usuario.java

```java
package com.sigcbqr.model.entity;

import jakarta.persistence.*;
import lombok.*;

import java.time.LocalDateTime;

@Entity
@Table(name = "usuarios")
@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
public class Usuario {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100)
    private String nombre;

    @Column(nullable = false, unique = true, length = 150)
    private String email;

    @Column(nullable = false)
    private String password;

    @Column(length = 20)
    private String telefono;

    private String foto;

    @Column(nullable = false)
    private Boolean activo;

    @ManyToOne(fetch = FetchType.EAGER)
    @JoinColumn(name = "rol_id", nullable = false)
    private Rol rol;

    @Column(name = "created_at")
    private LocalDateTime createdAt;

    @Column(name = "updated_at")
    private LocalDateTime updatedAt;

    @PrePersist
    protected void onCreate() {
        createdAt = LocalDateTime.now();
        updatedAt = LocalDateTime.now();
        if (activo == null) activo = true;
    }

    @PreUpdate
    protected void onUpdate() {
        updatedAt = LocalDateTime.now();
    }
}
```

db\procs\sp_crear_prestamo.sql

```sql
CREATE OR REPLACE PROCEDURE sp_crear_prestamo(
    IN p_usuario_id BIGINT,
    IN p_inventario_id BIGINT,
    IN p_dias_prestamo INT DEFAULT 7,
    INOUT p_prestamo_id BIGINT DEFAULT NULL
)
    LANGUAGE plpgsql
AS $$
DECLARE
    v_estado_inventario VARCHAR;
    v_prestamos_activos BIGINT;
BEGIN
    SELECT estado INTO v_estado_inventario FROM inventario WHERE id = p_inventario_id;
    IF v_estado_inventario IS NULL THEN
        RAISE EXCEPTION 'Ejemplar no encontrado';
    END IF;
    IF v_estado_inventario != 'DISPONIBLE' THEN
        RAISE EXCEPTION 'El ejemplar no está disponible';
    END IF;

    SELECT COUNT(*) INTO v_prestamos_activos
    FROM prestamos WHERE usuario_id = p_usuario_id AND estado = 'ACTIVO';
    IF v_prestamos_activos >= 5 THEN
        RAISE EXCEPTION 'El usuario tiene demasiados préstamos activos';
    END IF;

    INSERT INTO prestamos (usuario_id, inventario_id, fecha_prestamo, fecha_vencimiento, estado)
    VALUES (p_usuario_id, p_inventario_id, NOW(), NOW() + (p_dias_prestamo || ' days')::INTERVAL, 'ACTIVO')
    RETURNING id INTO p_prestamo_id;

    UPDATE inventario SET estado = 'PRESTADO' WHERE id = p_inventario_id;

    UPDATE libros l
    SET ejemplares_disponibles = ejemplares_disponibles - 1
    FROM inventario i
    WHERE i.id = p_inventario_id AND l.id = i.libro_id;
END;
$$;
```

db\procs\sp_devolver_prestamo.sql

```sql
CREATE OR REPLACE PROCEDURE sp_devolver_prestamo(
    IN p_prestamo_id BIGINT,
    INOUT p_multa_generada BOOLEAN DEFAULT FALSE
)
    LANGUAGE plpgsql
AS $$
DECLARE
    v_estado VARCHAR;
    v_inventario_id BIGINT;
    v_libro_id BIGINT;
    v_fecha_vencimiento TIMESTAMP;
    v_dias_retraso INT;
BEGIN
    SELECT estado, inventario_id, fecha_vencimiento
    INTO v_estado, v_inventario_id, v_fecha_vencimiento
    FROM prestamos WHERE id = p_prestamo_id;

    IF v_estado IS NULL THEN
        RAISE EXCEPTION 'Préstamo no encontrado';
    END IF;
    IF v_estado = 'DEVUELTO' THEN
        RAISE EXCEPTION 'El préstamo ya fue devuelto';
    END IF;

    UPDATE prestamos
    SET estado = 'DEVUELTO', fecha_devolucion = NOW()
    WHERE id = p_prestamo_id;

    UPDATE inventario SET estado = 'DISPONIBLE' WHERE id = v_inventario_id;

    UPDATE libros l
    SET ejemplares_disponibles = ejemplares_disponibles + 1
    FROM inventario i
    WHERE i.id = v_inventario_id AND l.id = i.libro_id;

    IF v_fecha_vencimiento < NOW() THEN
        v_dias_retraso := EXTRACT(DAY FROM (NOW() - v_fecha_vencimiento));
        INSERT INTO multas (prestamo_id, usuario_id, monto, pagada, fecha_multa)
        SELECT p_prestamo_id, p.usuario_id, v_dias_retraso * 0.50, false, NOW()
        FROM prestamos p WHERE p.id = p_prestamo_id;
        p_multa_generada := TRUE;
    END IF;
END;
$$;
```

db\procs\sp_reporte_prestamos_diarios.sql

```sql
CREATE OR REPLACE PROCEDURE sp_reporte_prestamos_diarios(
    IN p_fecha DATE,
    INOUT p_ref REFCURSOR
)
    LANGUAGE plpgsql
AS $$
BEGIN
    OPEN p_ref FOR
        SELECT p.id, p.fecha_prestamo, p.estado,
               u.nombre AS usuario_nombre,
               l.titulo AS libro_titulo
        FROM prestamos p
        JOIN usuarios u ON u.id = p.usuario_id
        JOIN inventario i ON i.id = p.inventario_id
        JOIN libros l ON l.id = i.libro_id
        WHERE p.fecha_prestamo::DATE = p_fecha
        ORDER BY p.fecha_prestamo;
END;
$$;
```